

Voice-Controlled Obstacle Avoiding Car

Code:

```
#include <Servo.h>

// ---- Servos ----

Servo frontServo; // servo that sweeps the front ultrasonic sensor
Servo rearServo;  // servo that sweeps the rear ultrasonic sensor


// ---- Ultrasonic Pins ----

const int frontTrig = 7; // front sensor: trigger pin
const int frontEcho = 8; // front sensor: echo pin
const int rearTrig = 12; // rear sensor: trigger pin
const int rearEcho = 13; // rear sensor: echo pin


// ---- Motor Pins ----

const int IN1 = 2; // right motor, forward
const int IN2 = 3; // right motor, backward
const int IN3 = 4; // left motor, forward
const int IN4 = 5; // left motor, backward


// ---- State ----

char command; // latest Bluetooth character received

bool goingForward = false; // true while car is moving forward
bool goingBackward = false; // true while car is moving backward
```

```

bool gentleTurning = false; // true during a gentle arc (not a sharp turn)

int  motorSpeed   = 180; // PWM duty for the outer/main wheel (0–255)

int  gentleFactor = 70;  // PWM duty for the inner wheel during arcs


// ---- Tuned Parameters ----

const int safeDistance = 45; // cm — trigger avoidance if object is closer
const int clearDistance = 65; // cm — path considered clear above this value
const int turnTime     = 420; // ms — how long one turn step runs
const int scanDelay    = 350; // ms — wait after servo.write() so servo settles


// ---- Distance check throttle ----

unsigned long lastDistCheck = 0;

const int  distCheckInterval = 150; // ms between quick obstacle polls in main
loop


// =====
// ULTRASONIC — single ping
// =====

int singlePing(int trigPin, int echoPin) {

    digitalWrite(trigPin, LOW);

    delayMicroseconds(4);    // ensure trigger starts clean


    digitalWrite(trigPin, HIGH);

```

```

delayMicroseconds(10);    // 10 µs pulse activates the sensor
digitalWrite(trigPin, LOW);

// 30 ms timeout ≈ 510 cm; returns 0 if nothing echoes in time
long duration = pulseIn(echoPin, HIGH, 30000);

// 65 ms gap lets echo fully decay before the next ping fires
delay(65);

if (duration == 0) return 999;    // timeout → treat as clear path
int dist = (int)(duration * 0.034 / 2); // µs → cm using speed of sound
if (dist < 2 || dist > 400) return 999; // discard out-of-range readings
return dist;
}

// =====
// ULTRASONIC — stable median of 5 pings
// (~325 ms total; only call when stopped or scanning)
// =====

int getDistance(int trigPin, int echoPin) {
    int readings[5];
    int valid = 0;

```

```
for (int i = 0; i < 5; i++) {  
    int d = singlePing(trigPin, echoPin);  
    if (d < 999) readings[valid++] = d; // collect only valid readings  
}
```

```
if (valid == 0) return 999; // all pings timed out → assume clear
```

```
// Bubble sort so we can pick the median
```

```
for (int i = 0; i < valid - 1; i++) {  
    for (int j = 0; j < valid - i - 1; j++) {  
        if (readings[j] > readings[j + 1]) {  
            int tmp    = readings[j];  
            readings[j] = readings[j + 1];  
            readings[j+1] = tmp;  
        }  
    }  
}
```

```
return readings[valid / 2]; // median filters out outlier spikes  
}
```

```
// =====
```

```
// ULTRASONIC — fast single reading for main loop polls
```

```
// =====
```

```

int quickDistance(int trigPin, int echoPin) {
    return singlePing(trigPin, echoPin); // one ping — fast, non-blocking feel
}

// =====
// MOTOR FUNCTIONS
// =====

void forward() {
    analogWrite(IN1, motorSpeed); digitalWrite(IN2, LOW); // right wheels: forward
    analogWrite(IN3, motorSpeed); digitalWrite(IN4, LOW); // left wheels: forward
}

void backward() {
    digitalWrite(IN1, LOW); analogWrite(IN2, motorSpeed); // right wheels:
backward
    digitalWrite(IN3, LOW); analogWrite(IN4, motorSpeed); // left wheels: backward
}

void turnRight() {
    analogWrite(IN1, motorSpeed); digitalWrite(IN2, LOW); // right wheels: forward
    digitalWrite(IN3, LOW); analogWrite(IN4, motorSpeed); // left wheels: backward
→ spins right
}

```

```
void turnLeft() {  
    digitalWrite(IN1, LOW); analogWrite(IN2, motorSpeed); // right wheels:  
    backward  
    analogWrite(IN3, motorSpeed); digitalWrite(IN4, LOW); // left wheels: forward →  
    spins left  
}
```

// Gentle arc: inner wheel slows down, outer wheel stays at full speed

```
void gentleRight() {  
    analogWrite(IN1, gentleFactor); digitalWrite(IN2, LOW); // right (inner): slow  
    analogWrite(IN3, motorSpeed); digitalWrite(IN4, LOW); // left (outer): fast  
}
```

```
void gentleLeft() {  
    analogWrite(IN1, motorSpeed); digitalWrite(IN2, LOW); // right (outer): fast  
    analogWrite(IN3, gentleFactor); digitalWrite(IN4, LOW); // left (inner): slow  
}
```

```
void stopMotors() {  
    // Pull all pins LOW — both wheels coast to a stop  
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);  
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);  
}
```

```

// =====
// SERVO — move to angle and wait for physical travel to finish
// =====

void servoGoto(Servo &s, int angle) {
    s.write(angle); // command the servo
    delay(scanDelay); // wait so the sensor is stable before pinging
}

// =====
// SCAN — sweep 5 angles and return the one with most clearance
// =====

int scanAllDirections(Servo &s, int trigPin, int echoPin) {
    const int angles[5] = {30, 60, 90, 120, 150}; // sweep left-to-right
    int    distances[5] = {0, 0, 0, 0, 0};

    Serial.println("Scanning...");

    for (int i = 0; i < 5; i++) {
        servoGoto(s, angles[i]);           // point sensor at this angle
        distances[i] = getDistance(trigPin, echoPin); // 5-sample median reading
        Serial.print(" "); Serial.print(angles[i]);
        Serial.print(" deg -> "); Serial.print(distances[i]); Serial.println(" cm");
    }

    servoGoto(s, 90); // return to centre after scan
}

```

```

// Find whichever angle gave the greatest clearance

int bestIndex = 2; // default to straight ahead (90°) if all equal

int bestDistance = 0;

for (int i = 0; i < 5; i++) {
    if (distances[i] > bestDistance) {
        bestDistance = distances[i];
        bestIndex = i;
    }
}

Serial.print("Best: "); Serial.print(angles[bestIndex]);
Serial.print(" deg ("); Serial.print(bestDistance); Serial.println(" cm)");
return angles[bestIndex];
}

// =====

// EXECUTE TURN based on best scan angle

// =====

void executeTurn(int bestAngle) {
    if (bestAngle < 75) { // best gap is to the right
        Serial.println("Turn RIGHT");
        turnRight(); delay(turnTime); stopMotors(); delay(200);
    } else if (bestAngle > 105) { // best gap is to the left
        Serial.println("Turn LEFT");
        turnLeft(); delay(turnTime); stopMotors(); delay(200);
    }
}

```

```

    } else {
        Serial.println("Ahead clear — no turn"); // 90° scan shows clear ahead
    }
}

// =====

// AVOID OBSTACLE — FRONT

// =====

void avoidObstacleForward() {
    Serial.println("=== OBSTACLE FRONT ===");

    goingForward = false; // pause forward-motion flag so main loop doesn't re-
    check

    stopMotors();

    delay(300);

    // Quick left/right peek before committing to full 5-point scan

    servoGoto(frontServo, 150);

    int leftDist = getDistance(frontTrig, frontEcho);

    Serial.print("Quick left: "); Serial.println(leftDist);

    servoGoto(frontServo, 30);

    int rightDist = getDistance(frontTrig, frontEcho);

    Serial.print("Quick right: "); Serial.println(rightDist);

    servoGoto(frontServo, 90); // centre sensor again

```

```
// Take the obviously clear side if one side has a big advantage
if (leftDist > clearDistance && leftDist > rightDist + 20) {
    Serial.println("Left clear — turning LEFT");
    turnLeft(); delay(turnTime); stopMotors(); delay(200);
    goingForward = true; forward(); return;
}

if (rightDist > clearDistance && rightDist > leftDist + 20) {
    Serial.println("Right clear — turning RIGHT");
    turnRight(); delay(turnTime); stopMotors(); delay(200);
    goingForward = true; forward(); return;
}


// Full scan loop — up to 6 attempts before giving up
for (int attempt = 1; attempt <= 6; attempt++) {
    int bestAngle = scanAllDirections(frontServo, frontTrig, frontEcho);
    executeTurn(bestAngle);
    delay(300);

    // Verify the path after turning
    servoGoto(frontServo, 90);
    int checkDist = getDistance(frontTrig, frontEcho);
    Serial.print("Path check: "); Serial.println(checkDist);
}
```

```
if (checkDist > clearDistance) {  
    Serial.println("Path CLEAR — resuming FORWARD");  
    goingForward = true; forward(); return;  
}  
  
Serial.print("Attempt "); Serial.print(attempt); Serial.println(" — still blocked");  
  
// Every 2 failed turns, reverse briefly to get a new vantage point  
if (attempt % 2 == 0) {  
    Serial.println("Backing up to reposition");  
    backward(); delay(550); stopMotors(); delay(300);  
}  
}  
  
// Last resort: longer reverse then final scan  
Serial.println("Stuck — longer reverse");  
backward(); delay(1000); stopMotors(); delay(400);  
int finalAngle = scanAllDirections(frontServo, frontTrig, frontEcho);  
executeTurn(finalAngle);  
delay(300);  
goingForward = true;  
forward();  
}
```

```
// =====
// AVOID OBSTACLE — REAR (mirrors forward logic)
// =====

void avoidObstacleBackward() {
  Serial.println("=== OBSTACLE REAR ===");
  goingBackward = false; // pause backward flag
  stopMotors();
  delay(300);

  // Quick peek with the rear servo
  servoGoto(rearServo, 150);
  int leftDist = getDistance(rearTrig, rearEcho);
  Serial.print("Quick left: "); Serial.println(leftDist);

  servoGoto(rearServo, 30);
  int rightDist = getDistance(rearTrig, rearEcho);
  Serial.print("Quick right: "); Serial.println(rightDist);

  servoGoto(rearServo, 90);

  if (leftDist > clearDistance && leftDist > rightDist + 20) {
    Serial.println("Left clear — turning LEFT");
    turnLeft(); delay(turnTime); stopMotors(); delay(200);
  }
}
```

```

    goingBackward = true; backward(); return;
}

if (rightDist > clearDistance && rightDist > leftDist + 20) {
    Serial.println("Right clear — turning RIGHT");
    turnRight(); delay(turnTime); stopMotors(); delay(200);
    goingBackward = true; backward(); return;
}

for (int attempt = 1; attempt <= 6; attempt++) {
    int bestAngle = scanAllDirections(rearServo, rearTrig, rearEcho);
    executeTurn(bestAngle);
    delay(300);

    servoGoto(rearServo, 90);
    int checkDist = getDistance(rearTrig, rearEcho);
    Serial.print("Rear path check: "); Serial.println(checkDist);

    if (checkDist > clearDistance) {
        Serial.println("Rear CLEAR — resuming BACKWARD");
        goingBackward = true; backward(); return;
    }

    Serial.print("Attempt "); Serial.print(attempt); Serial.println(" — still blocked");
}

```

```
// Drive forward briefly to reposition when reversing is blocked
```

```
if (attempt % 2 == 0) {
```

```
    Serial.println("Moving forward to reposition");
```

```
    forward(); delay(550); stopMotors(); delay(300);
```

```
}
```

```
}
```

```
Serial.println("Stuck — longer forward push");
```

```
forward(); delay(1000); stopMotors(); delay(400);
```

```
int finalAngle = scanAllDirections(rearServo, rearTrig, rearEcho);
```

```
executeTurn(finalAngle);
```

```
delay(300);
```

```
goingBackward = true;
```

```
backward();
```

```
}
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup() {
```

```
    // Motor driver pins all need to be outputs
```

```
    pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
```

```
    // Ultrasonic: trigger = output, echo = input
```

```

pinMode(frontTrig, OUTPUT); pinMode(frontEcho, INPUT);
pinMode(rearTrig, OUTPUT); pinMode(rearEcho, INPUT);

// Attach servos and centre them before any movement begins
frontServo.attach(6); rearServo.attach(9);
frontServo.write(90); rearServo.write(90);
delay(800); // physical settle time — servos need this before first ping

stopMotors();
Serial.begin(9600);

Serial.println("=== System Ready ===");
Serial.println("F/B=Forward/Backward L/R=Turn G/H=Gentle turn S=Stop");
Serial.println("1-5=Speed");
}
// =====
// MAIN LOOP
// =====
void loop() {

// ---- Read incoming Bluetooth character ----
if (Serial.available()) {
    command = Serial.read();

```

```
Serial.print("CMD: "); Serial.println(command);

// Clear all movement flags so old state doesn't linger
goingForward = false;
goingBackward = false;
gentleTurning = false;

if (command == 'F' || command == 'f') { goingForward = true; forward();
Serial.println("FORWARD"); }

else if (command == 'B' || command == 'b') { goingBackward = true; backward();
Serial.println("BACKWARD"); }

else if (command == 'L' || command == 'l') { turnLeft(); delay(500);
stopMotors(); Serial.println("TURN LEFT"); }

else if (command == 'R' || command == 'r') { turnRight(); delay(500);
stopMotors(); Serial.println("TURN RIGHT"); }

// G/H: gentle arcs — still count as goingForward for front-sensor checks

else if (command == 'G' || command == 'g') { goingForward = true;
gentleTurning = true; gentleRight(); Serial.println("GENTLE RIGHT"); }

else if (command == 'H' || command == 'h') { goingForward = true;
gentleTurning = true; gentleLeft(); Serial.println("GENTLE LEFT"); }

else if (command == 'S' || command == 's') { stopMotors();
Serial.println("STOP"); }
```

```
// Speed presets — gentleFactor scales with motorSpeed to keep arc shape consistent
```

```
    else if (command == '1') { motorSpeed = 80; gentleFactor = 30;
    Serial.println("Speed 1 — very slow"); }
```

```
    else if (command == '2') { motorSpeed = 120; gentleFactor = 45;
    Serial.println("Speed 2 — slow"); }
```

```
    else if (command == '3') { motorSpeed = 180; gentleFactor = 70;
    Serial.println("Speed 3 — medium"); }
```

```
    else if (command == '4') { motorSpeed = 220; gentleFactor = 90;
    Serial.println("Speed 4 — fast"); }
```

```
    else if (command == '5') { motorSpeed = 255; gentleFactor = 110;
    Serial.println("Speed 5 — full"); }
```

```
}
```

```
// ---- Throttled obstacle checks (one quick ping per interval) ----
```

```
// quickDistance() is used here — one ping every 150 ms keeps the
```

```
// command reader responsive while still catching obstacles in time.
```

```
if (millis() - lastDistCheck >= distCheckInterval) {
```

```
    lastDistCheck = millis(); // reset the timer
```

```
if (goingForward) {
```

```
    int frontDist = quickDistance(frontTrig, frontEcho);
```

```
    if (frontDist != 999) { Serial.print("Front: "); Serial.println(frontDist); } // skip
logging 999 (timeout)
```

```
    if (frontDist > 0 && frontDist < safeDistance) {
```

```
    avoidObstacleForward();

    gentleTurning = false; // avoidance always ends going straight — clear the arc
flag
    }
}

if (goingBackward) {
    int rearDist = quickDistance(rearTrig, rearEcho);
    if (rearDist != 999) { Serial.print("Rear: "); Serial.println(rearDist); }
    if (rearDist > 0 && rearDist < safeDistance) {
        avoidObstacleBackward();
    }
}
}
}
```